

Product Requirements Document

Offline Field Issue Tracker

Document owner	Product and engineering
Version	1.0
Date	2026-10-02
Status	Assessment-ready
Primary audience	Product, design, engineering, QA, assessors

Product promise: A field worker can capture a complete infrastructure issue without connectivity, trust that it is durably stored, and see it reach a coordinator exactly once when connectivity returns.

1. Product Vision and Problem Statement

Vision

Build a dependable, offline-first issue reporting system for infrastructure teams working in tunnels, rural areas, basements, construction sites, and other connectivity-constrained environments. The product makes local capture the source of truth until the server acknowledges a synchronized record, while giving coordinators a clear, auditable operational queue.

Problem

Field workers currently rely on paper, memory, chat messages, or forms that fail when the network is slow or unavailable. This causes:

- Lost or duplicated reports after a failed submission.
- Incomplete location and evidence data.
- Delayed triage and dispatch.
- Unclear ownership and status history.
- Poor confidence that a report was received.

The system must optimize for **durability before immediacy**: local persistence and recoverability are more important than a fast-looking network request.

Success measures

Measure	Target
Local save success	100% of valid submissions while browser storage is available
Duplicate server records from retries	0 for the same client-generated report ID
Local list/filter interaction	p95 under 1 second for 1,000 cached reports

Measure	Target
Sync recovery	Automatic retry when connectivity returns, with manual retry available
Audit completeness	Every server-side status transition has actor, time, old state, new state, and note
Understandability	A worker can distinguish saved locally from server-synced without technical knowledge

2. Personas

Field Worker — offline-first data collector

Attribute	Requirement
Environment	Mobile browser, gloves, bright sun or low light, intermittent/no network
Goals	Record an issue quickly, attach accurate location, avoid duplicate work, continue safely
Frustrations	Spinners that never finish, forms that reset, unclear sync status, tiny controls
Core permissions	Create reports, edit drafts, submit reports, view own queue, retry failed syncs
Success signal	“Saved on this device” is explicit even when offline

Coordinator — reviewer, triager, dispatcher

Attribute	Requirement
Environment	Desktop browser, stable network, many concurrent reports
Goals	Filter and prioritize issues, assign work, reject bad reports with reasons, audit outcomes
Frustrations	Duplicates, missing context, hidden conflicts, status changes without history
Core permissions	Review all reports, transition workflow states, add notes, inspect history
Success signal	A trustworthy queue with chronology and actionable next states

3. Scope

In scope

1. Report capture with category, description, priority, location, and timestamps.
2. Draft persistence and submitted-report persistence in IndexedDB.
3. Client-generated UUIDs and idempotent batch or individual synchronization.
4. Connectivity observation, automatic retry, queue visibility, and manual retry.
5. Coordinator filtering, review, rejection, assignment, resolution, and history.
6. Server-side workflow validation and append-only history.
7. Explicit error presentation and recovery actions.

Out of scope for the assessment

- Native mobile applications and background OS sync.
- Binary photo/video upload, image processing, and document storage.
- Push notifications, SMS, email, or external dispatch integrations.
- Multi-tenant billing, SSO, advanced RBAC, and analytics.
- Geospatial maps, route optimization, or device fleet management.

4. Functional Requirements

FR-01: Report creation

- The worker can create a draft with required **category**, **description**, **priority**, and **location**.
- Location is captured from GPS when permission and a usable fix exist; manual entry is always available.
- The form validates locally before allowing submission.
- The worker can save a draft explicitly or navigate away without losing entered data.
- Submission assigns a UUID at first creation and never changes it.

FR-02: Offline persistence

- Drafts and submitted reports are stored in IndexedDB through Dexie.js.
- A successful local transaction must complete before the UI reports "Saved".
- The UI shows one of **Draft**, **Pending Sync**, **Synced**, or **Failed**.
- Storage errors are visible and actionable; the app must not present a success-shaped fallback.

FR-03: Sync engine

- The sync queue contains unsynced records, their retry count, last error, and next attempt time.
- The engine observes **online/offline** events and performs a bounded retry with exponential backoff and jitter.
- Each request sends **Idempotency-Key: <stable-key>** and the report UUID in the payload.
- A successful response marks the local record **Synced** and stores **client_synced_at**.
- A network error retains the item in the queue.
- A 422 validation error marks the item **Failed** and requires correction.
- A 409 conflict presents the server version and preserves the local audit context.
- A 500 response remains retryable and is visible as a server failure.

FR-04: Coordinator operations

- Coordinators can filter by status, priority, category, and date.
- Coordinators can transition valid reports through the workflow state machine.
- Rejection requires a reason.
- Assignment requires an assignee identifier in the implementation layer, even if the assessment UI uses a simple text selector.
- Every transition creates a **report_history** row.

FR-05: Audit log

- The system records report creation, synchronization attempts, status transitions, rejection reasons, and conflict outcomes.
- History is chronological, immutable from the UI, and available through `GET /api/reports/:id/history`.
- A retry must not create a second business transition or duplicate report.

5. Non-Functional Requirements

ID	Requirement	Acceptance criterion
NFR-01	Zero data loss	A confirmed local save survives reload, tab close, and temporary offline periods
NFR-02	Idempotency	Replaying a request with the same idempotency key returns the original outcome
NFR-03	Local performance	Cached list/filter queries complete within 1 second at p95 for 1,000 records
NFR-04	Recovery	Sync resumes on reconnect and never discards a queue item because a retry failed
NFR-05	Consistency	Server rejects invalid transitions atomically; report and history update together
NFR-06	Accessibility	Keyboard operable; visible focus; controls at least 48×48 CSS px; WCAG AA contrast target
NFR-07	Observability	Sync attempts have structured logs with report ID, idempotency key, result class, and duration
NFR-08	Security baseline	Validate payloads server-side, use HTTPS in deployment, and avoid storing secrets in IndexedDB

6. Ambiguities Identified and Decisions

Ambiguity	Decision
Can a worker edit after submission?	Yes, while Pending Sync ; after Synced , edits create a controlled update flow rather than mutating history silently
What does “offline” mean?	Browser connectivity signal plus actual request failure; online is only a hint, not proof of server reachability
Who may reject?	Coordinator role only; server authorization is authoritative
How are duplicate issues handled?	Do not auto-merge. A coordinator may reject one as duplicate with a note and linked report reference in a later extension
What happens when GPS is unavailable?	Require manual location with an explicit “GPS unavailable/manual” source

Ambiguity	Decision
What is the conflict policy?	Last-Write-Wins for mutable report fields, preserving every server transition and conflict decision in audit history
Is a resolved issue reopened?	Default behavior is a linked follow-up; reopening requires an explicit coordinator action and a reason

7. Edge-Case Handling

1. **Double tap on Submit:** disable the action during the local transaction; UUID and idempotency key make a repeated request safe.
2. **Reload during sync:** queue state is durable; the next app boot resumes pending items.
3. **Connectivity flaps:** the worker processes one item at a time per report and applies backoff; no tight retry loop.
4. **Clock skew:** server timestamps are authoritative for server events; client time is retained as capture metadata.
5. **Storage quota exceeded:** stop accepting new local data, show the error, and guide the worker to sync or remove completed local records.
6. **GPS permission denied:** show manual location fields and record the source as **manual**.
7. **Malformed server response:** classify as server failure, retain the queue item, and log a parse error.
8. **Invalid workflow transition:** return 409 with current state and allowed transitions; do not write history.
9. **Resolved report needing more work:** create a linked follow-up by default to preserve the resolved record's audit trail.
10. **Partial batch result:** return per-item outcomes; successful items are marked synced while retryable failures remain queued.

8. Release Acceptance Checklist

- ☐ Create and save a draft in offline mode.
- ☐ Reload and verify the draft remains.
- ☐ Submit offline and verify **Pending Sync**.
- ☐ Reconnect and verify automatic sync and **Synced**.
- ☐ Replay the same request and verify no duplicate report.
- ☐ Execute every allowed and forbidden workflow transition.
- ☐ Verify rejection requires and stores a reason.
- ☐ Verify chronological history includes all transitions.